

Today's Agenda

Motivation

Architecture of CNN

Parameters of Conv layer

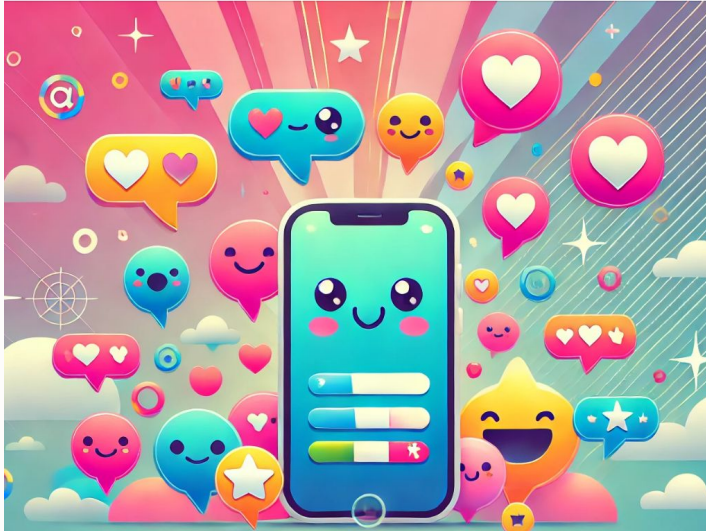
Q&A

Convolutional neural networks

Motivation

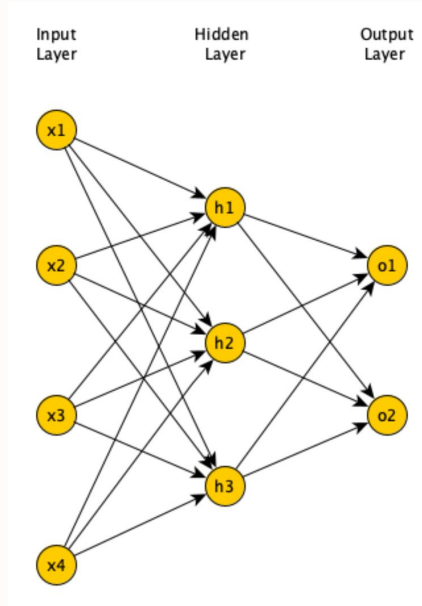
Motivation - New challenges

- Increasing popularity of social media in past decade
- Image and video processing tasks have become very important
- FNN could not scale up to image and video processing tasks
- CNN specifically tailored for image and video processing tasks



Motivation - FNN

- In FNN all nodes in a layer connected to all nodes in next layer
- Each connection has a weight, must be learned by learning algorithm



Motivation - Limitations of FNN

- If the input is a 64 by 64 pixel grayscale image:
 - Each grayscale pixel is represented by 1 value, usually between 0 and 255,
 - where 0 is black, 255 is white, and values in between are shades of gray.
- Since each grayscale pixel is represented by 1 value, we say the channel size is 1.
- The image is represented by $64 \times 64 \times 1 = 4,096$ values (rows $\times$ columns $\times$ channels).
 - Hence, the input layer of the FNN has 4,096 nodes.
- Let's assume the next layer has 500 nodes.
 - Since the FNN is fully connected, we have $4,096 \times 500 = 2,048,000$ weights.

Motivation - Limitations of FNN

For complex problems, we need multiple hidden layers in our FNN.

This **compounds** the problem of having many weights.

Having too many weights:

- Makes learning more difficult, as the dimensionality of the search space increases
- Makes training more time/resource-consuming
- Increases the likelihood of overfitting

Motivation - Limitations of FNN

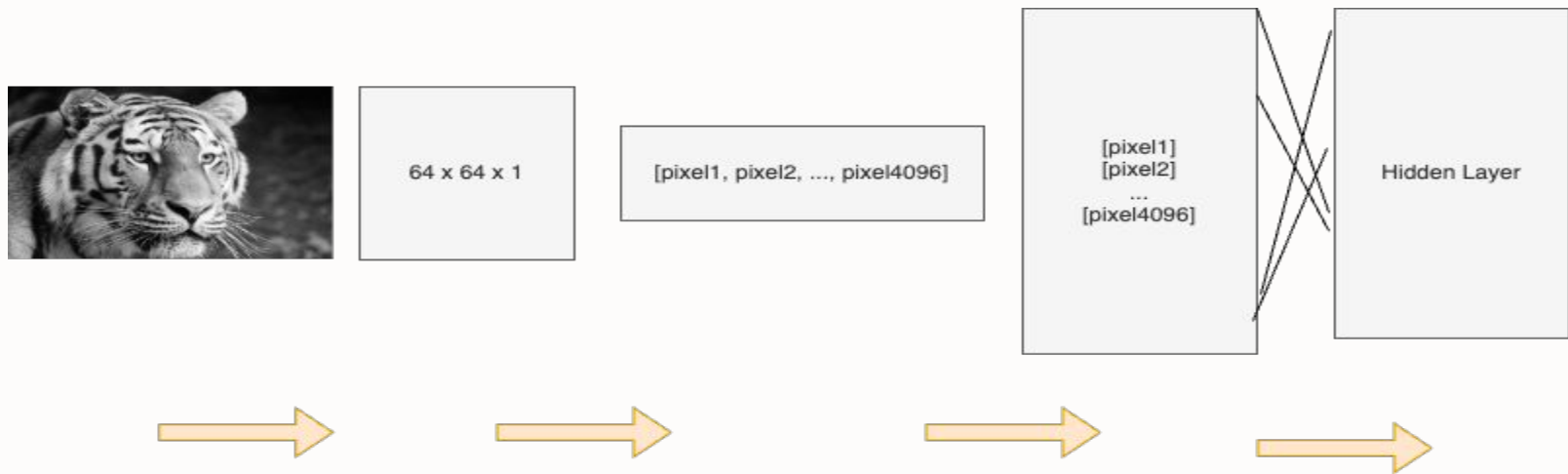
The problem is further **compounded** for color images.

- Each pixel in a color image is represented by 3 values (RGB color mode).
- Since each pixel is represented by 3 values, we say the channel size is 3.
- The image is represented by $64 \times 64 \times 3 = 12,288$ values (rows $\times$ columns $\times$ channels).
- The number of weights is now $12,288 \times 500 = 6,144,000$.

Motivation - Limitations of FNN

- It's clear that FNNs cannot scale well to larger images due to the excessive number of weights.
- Another problem with FNNs is that a 2D image is represented as a 1D vector in the input layer.
- As a result, any spatial relationships in the data are ignored.

FNN Pipeline

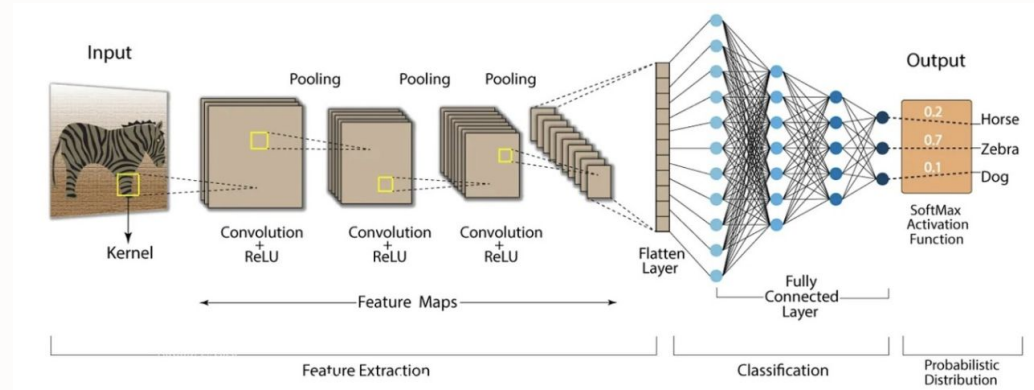


Architecture of CNN

Architecture of CNN

A typical CNN has four layers:

- Input layer
- Convolutional layer
- Pooling layer
- Fully connected layer



We will focus on 2D CNN

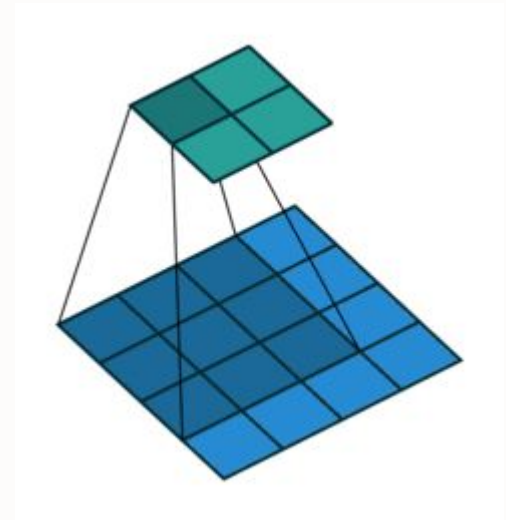
The same concepts apply to 1D or 3D CNNs as well.

Input layer

- **Example:** Input is a 28 by 28 pixel grayscale image.
- Unlike in an FNN, we do not “flatten” the input into a 1D vector.
 - Instead, the input is presented to the network as a 2D matrix: 28×28 .
 - This allows the model to better capture spatial relationships in the image.

Convolutional layer

- A convolutional layer uses multiple 2D filters (kernels).
- For example, a 3×3 filter (with 9 values, often initialized randomly) slides over the image.
- At each position, it multiplies its values with the overlapping pixel values and sums the result.
- The filter moves right one pixel at a time, and when it reaches the edge, it shifts down and continues.



Parameters of Convolutional layer

Convolutional layer parameters

- Filter size
- Padding
- Stride
- Dilation
- Activation function

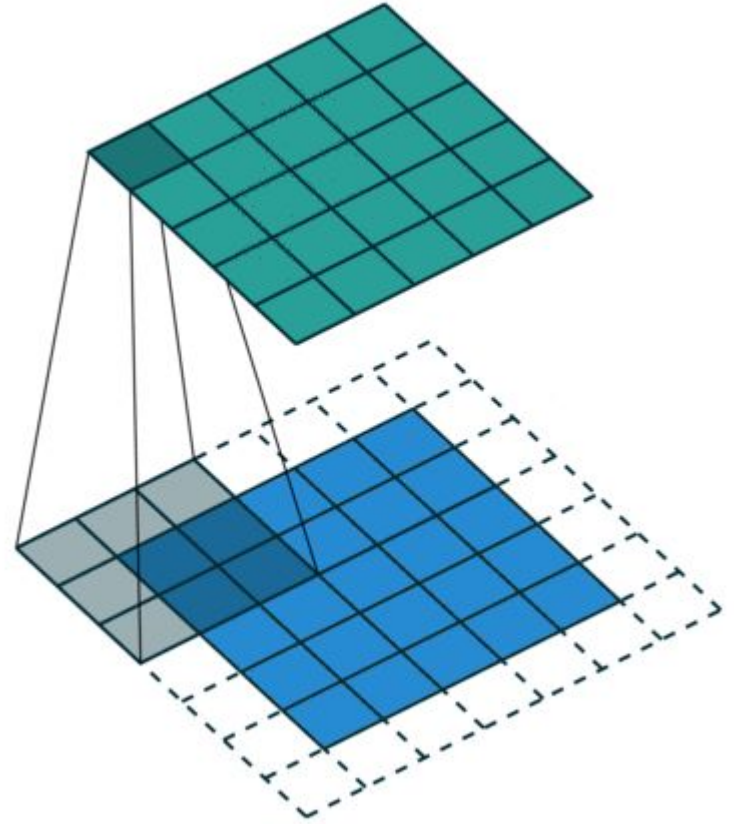
Filter size

- Filter size can be 5 by 5, 3 by 3, and so on
- Larger filter sizes should be avoided
- Odd sized filters are preferred to even sized filters

Padding

- After applying 3 by 3 filter to 4 by 4 image, we get a 2 by 2 image – Size of the image has gone down
- If we want to keep image size the same, we can use padding
 - We pad input in every direction with 0's before applying filter
 - If padding is 1 by 1, then we add 1 zero in every direction
 - If padding is 2 by 2, then we add 2 zeros in every direction, and so on

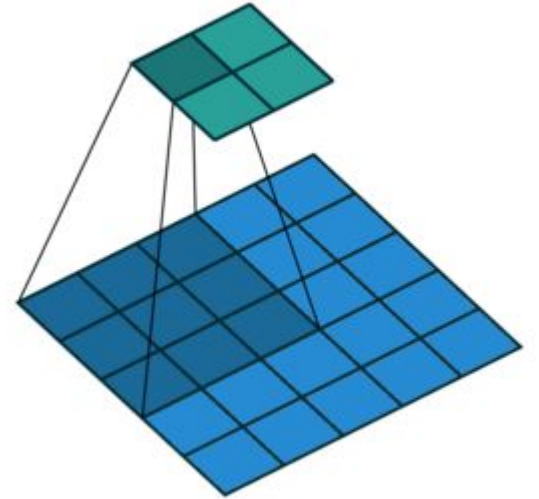
Img: 3x3 filter with padding of 1



Stride

The number of pixels the filter moves to the right or down is called the **stride**.

- **Stride 1**: the filter moves one pixel at a time
- **Stride 2**: the filter moves two pixels at a time

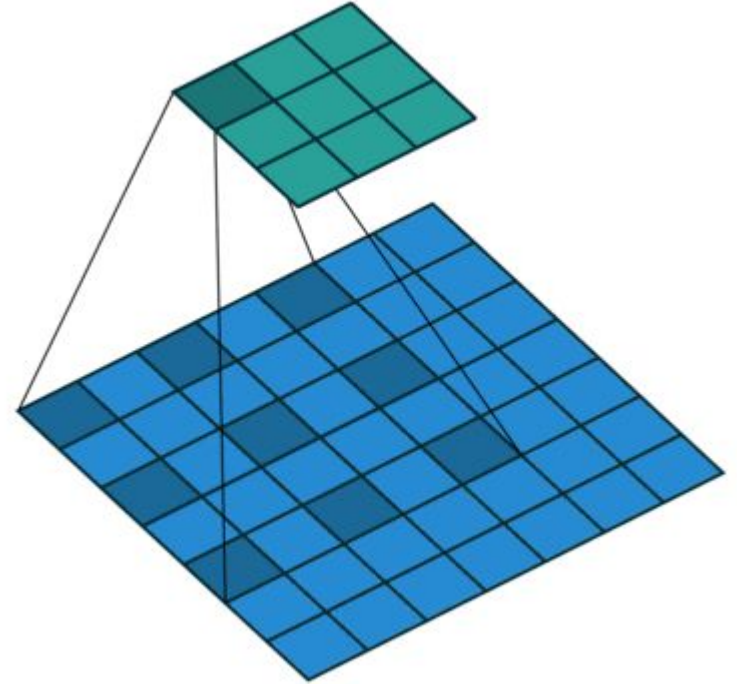


Img: 3x3 filter with stride of 2

Dilation

- When a 3×3 filter is applied, the output is influenced by a 3×3 region of the image.
- **Dilation** increases the receptive field—the portion of the image that affects the output.
- For example, with a **dilation rate of 2**, the filter skips every other pixel, so a 5×5 area contributes to the output instead of a contiguous 3×3 region.

Img: 3x3 filter with dilation of 2



Activation function

- After the filter is applied to the entire image, an **activation function** is used to introduce non-linearity.
- The most common activation function in CNNs is **ReLU (Rectified Linear Unit)**.
- ReLU keeps positive values unchanged and replaces negative values with 0.

Relu activation function

0	1	-4
0	2	0
-1	4	3

Filter output

→

0	1	0
0	2	0
0	4	3

Filter output after ReLU

Example: Single channel 2D convolution

0	2	1	0	1
0	1	2	1	0
1	0	2	0	0
1	0	0	1	1
0	1	1	2	2

Input Vector

0	1	-4
0	2	0
-1	4	3

Filter

5		

Output

$$\begin{aligned} &= 0 \times 0 + 2 \times 1 + 1 \times -4 + \\ &\quad 0 \times 0 + 1 \times 2 + 2 \times 0 + \\ &\quad 1 \times -1 + 0 \times 4 + 2 \times 3 = 5 \end{aligned}$$

Example: Triple channel 2D convolution

0	2	1	1	1
0	1	2	1	0
1	0	2	0	0
1	1	0	1	0
1	0	1	0	2

Input Vector (Channel 1)

0	1	-4
0	2	0
-1	4	3

Channel 1 filter

5		

Channel 1 output

0	0	1	0	1
1	0	0	1	0
0	1	1	0	1
1	3	0	1	0
0	1	1	1	2

Input Vector (Channel 2)

0	1	0
1	1	0
-2	0	3

Channel 2 filter

4		

Channel 2 output

19		

Output

2	3	0	1	1
0	0	1	1	0
2	0	1	3	0
1	0	3	0	1
0	2	0	2	2

Input Vector (Channel 3)

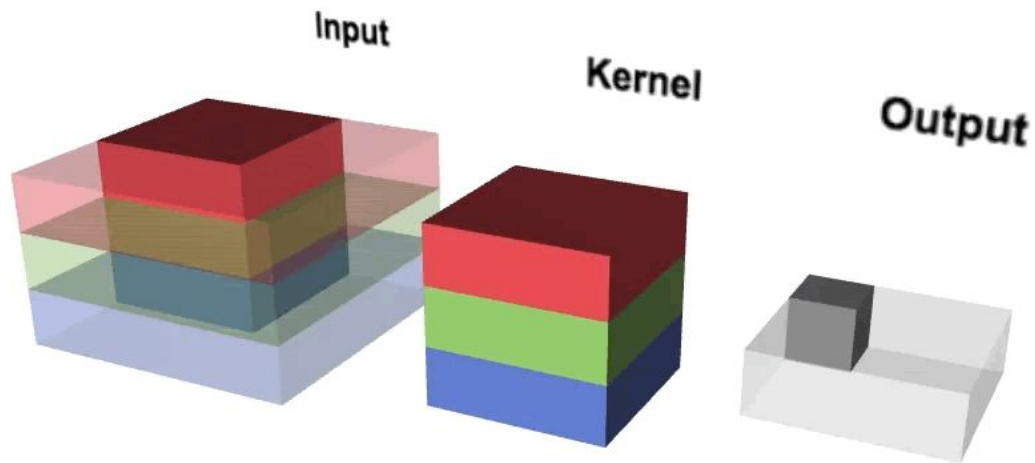
1	1	-4
0	0	0
4	1	-3

Channel 3 filter

10		

Channel 3 output

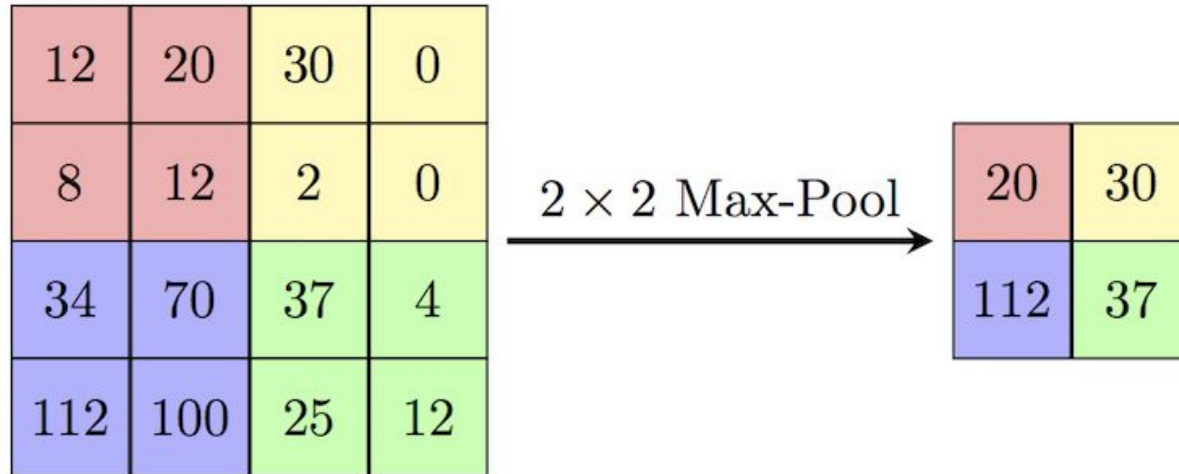
Example: Triple channel 2D convolution



Architecture of CNN (Continuation)

Pooling layer

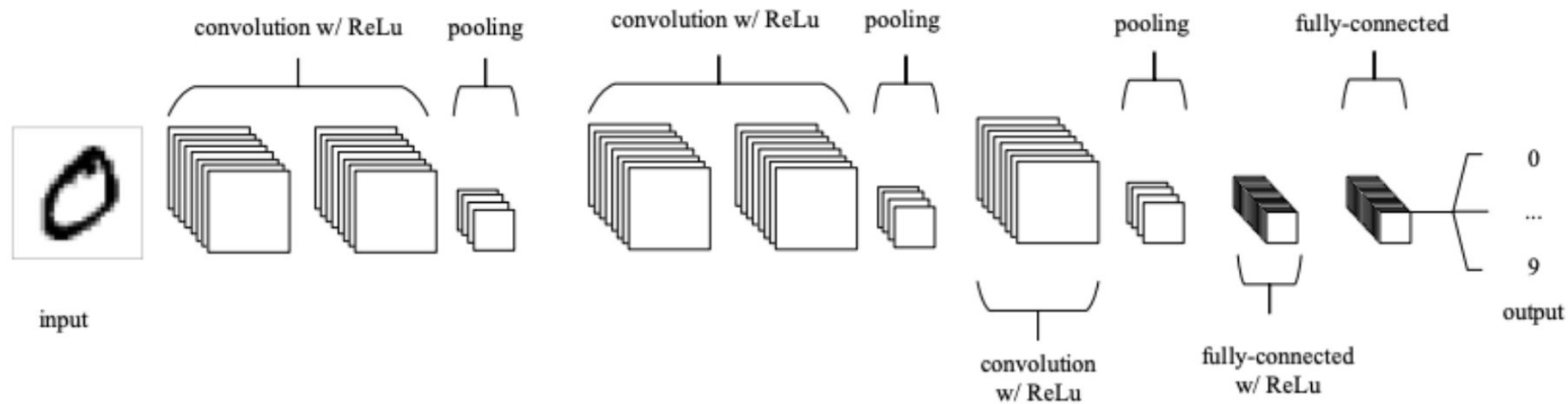
- The **pooling layer** downsamples the input to reduce spatial dimensions, which lowers the number of parameters, speeds up training, and helps prevent overfitting.
- The most common type is **max pooling**, typically using a 2×2 filter with a stride of 2, which returns the maximum value in each region as it slides over the input.



Fully connected layer

- Last layer in a CNN
- Connects all nodes from previous layer to this fully connected layer

An example of CNN



Q&A time!